

Taller: Librerías de Python para inteligencia artificial

Análisis del dataset Netflix TV Shows and Movies (titles.csv) con NumPy, SciPy, Matplotlib y Seaborn

Curso: Fundamentos para Inteligencia Artificial (NRC 94103)

Institución: Universidad Minuto de Dios (UNIMINUTO)

Docente: Ing. César Alfonso Bolado Silva

Actividad: Semana 4 - Taller colaborativo

Elaborado por: _____

Fecha de entrega: Julio de 2026

1. Reconocimiento de frameworks de IA en Python

Un framework es una estructura de trabajo que impone una arquitectura y un flujo determinado al programa (por ejemplo, definir las capas de una red neuronal siguiendo reglas propias del framework), mientras que una librería es un conjunto de funciones reutilizables que se invocan libremente, sin imponer una estructura al programa completo.

Cuatro frameworks/librerías comunes en inteligencia artificial:

- **Scikit-learn:** librería de machine learning clásico para clasificación, regresión, clustering y preprocesamiento de datos.
- **TensorFlow:** framework para construir y entrenar redes neuronales y modelos de deep learning a gran escala.
- **PyTorch:** framework de deep learning flexible, muy usado en investigación, visión por computador y procesamiento de lenguaje natural.
- **Hugging Face (Transformers):** librería/ecosistema de modelos preentrenados para procesamiento de lenguaje natural (NLP) y otras tareas.

Estas herramientas se usan en tareas de clasificación, regresión, visión por computador, procesamiento de lenguaje natural (NLP) y clustering, entre otras.

2. Selección, carga y preparación del dataset

Se utilizó el dataset Netflix TV Shows and Movies (Soeiro, 2022), específicamente el archivo titles.csv, que describe el catálogo de títulos (películas y series) disponibles en Netflix hasta 2022, con metadatos de producción y métricas externas de calificación (IMDb) y popularidad (TMDB).

2.1 Estructura original

El archivo se distribuye comprimido en formato ZIP, por lo que se cargó con `pandas.read_csv(..., compression="zip")`. `df.shape` reporta 5 850 filas y 15 columnas; `df.info()` confirma los tipos: 7 columnas de texto, 2 enteras y 5 numéricas de tipo decimal (float), con distintos niveles de valores nulos.

Variable	Descripción
id	Identificador único del título en la plataforma
title	Nombre del título
type	Tipo de contenido: MOVIE o SHOW
description	Sinopsis del título
release_year	Año de estreno
age_certification	Clasificación por edades (TV-MA, PG-13, etc.)
runtime	Duración en minutos (por episodio, en series)
genres	Lista de géneros asociados (texto con formato de lista de Python)
production_countries	Lista de países de producción (texto con formato de lista de Python)
seasons	Número de temporadas (solo aplica a series)
imdb_id	Identificador del título en IMDb
imdb_score	Calificación promedio en IMDb (0-10)
imdb_votes	Número de votos recibidos en IMDb
tmdb_popularity	Índice de popularidad en TMDB
tmdb_score	Calificación promedio en TMDB (0-10)

2.2 Calidad de datos y decisiones de limpieza

- **title** tiene 1 valor nulo: al ser un único registro sin nombre y sin forma de identificarlo, se eliminó esa fila (`dropna`).
- **age_certification** tiene 2 619 valores nulos (44.8% de las filas): es una proporción demasiado alta para eliminar registros, así que se imputó con la categoría "No clasificado", preservando la observación en vez de descartarla.
- **seasons** tiene 3 744 valores nulos, que corresponden exactamente al número de películas del catálogo: no es un dato faltante por error de captura, sino estructural (solo las series tienen temporadas), por lo que se imputó con 0 en las películas.
- **imdb_score**, **imdb_votes**, **tmdb_popularity** y **tmdb_score** tienen entre 91 y 498 nulos (1.6%-8.5%): se asumió ausencia aleatoria (títulos aún no calificados por la comunidad) y se imputaron con la mediana de cada variable, por ser una medida robusta frente a distribuciones asimétricas y para no perder observaciones en los cálculos posteriores.

- **runtime == 0** apareció en 14 filas: una duración de 0 minutos es una inconsistencia de captura, no un valor real. Se trataron como faltantes y se imputaron con la mediana de runtime según el type (película o serie), pues la duración típica difiere bastante entre ambos formatos.
- Duplicados: `df["id"].duplicated().sum()` reportó 0 filas duplicadas.
- `imdb_id` y `description` no se usaron en los cálculos numéricos de este taller, por lo que sus nulos no requirieron tratamiento.

2.3 Variables seleccionadas para el análisis

- **imdb_score** (numérica): calificación promedio en IMDb, variable central de calidad percibida.
- **tmdb_popularity** (numérica): índice de popularidad en TMDB, variable central de alcance/consumo.
- **type** (categórica): MOVIE o SHOW.
- **primary_genre** (categórica derivada): primer género de la lista original en genres, extraído con `ast.literal_eval`, ya que la columna original llega como texto con formato de lista de Python (por ejemplo `"['drama', 'action']"`).

Como apoyo se conservaron también `runtime` y `release_year` para los cálculos de correlación y el mapa de calor.

2.4 Tratamiento de valores atípicos

`tmdb_popularity` presenta una cola larga: el percentil 99 se ubica en 273.64, y 59 títulos (1.0% del catálogo) superan ese valor, con un máximo cercano a 2 274. Estos valores no son errores de captura, sino títulos genuinamente muy vistos a nivel global, por lo que no se eliminaron; se conservó la variable original para los cálculos y las pruebas estadísticas, y se creó una versión recortada al percentil 99 (`tmdb_popularity_clip`) únicamente para que las gráficas de dispersión y regresión fueran legibles.

3. Implementación con NumPy (cálculo y transformación)

Se aplicaron cuatro acciones de NumPy sobre las variables numéricas seleccionadas:

- 1) Conversión de `imdb_score` y `tmdb_popularity` a arreglos de NumPy (`np.array`), de 5 849 elementos cada uno.
- 2) Estadísticos descriptivos: IMDb score -> media = 6.518, mediana = 6.6, percentiles [25, 75] = [5.9, 7.3]. TMDB popularidad -> media = 22.395, mediana = 6.821, percentiles [25, 75] = [2.765, 16.244].
- 3) Operación vectorizada sin ciclos: estandarización z-score manual de `imdb_score`, $z = (x - \text{media}) / \text{desviación estándar}$; se identificaron 248 títulos con $|z| > 2$ (calificación inusualmente alta o baja).

- 4) np.unique con conteos sobre las variables categóricas: distribución por type -> 3 743 MOVIE (64.0%) y 2 106 SHOW (36.0%); top 5 de primary_genre -> drama (1 421), comedy (1 305), documentation (665), thriller (377) y action (365).

Interpretación: el catálogo tiene casi el doble de películas que de series. El puntaje IMDb promedio (6.518) es ligeramente menor que la mediana (6.6), lo que sugiere una leve asimetría hacia la izquierda (algunos títulos con calificación muy baja arrastran el promedio). La popularidad en TMDb, en cambio, tiene una media (22.395) muy superior a su mediana (6.821), señal clara de la cola larga hacia la derecha que se detectó al tratar los atípicos. El z-score vectorizado permitió, sin ciclos for, ubicar 248 títulos con calificación estadísticamente inusual.

4. Implementación con SciPy (análisis / estadística)

Se aplicaron los siguientes procedimientos de scipy.stats:

- Prueba de normalidad (Shapiro-Wilk) sobre una muestra de 5 000 valores de imdb_score: estadístico = 0.9719, $p < 0.001$. Se rechaza la normalidad.
- Correlación de Pearson entre imdb_score y tmdb_popularity: $r = 0.0166$, $p = 0.2057$ (no significativa al 5%).
- Prueba t de Welch comparando imdb_score entre MOVIE (media = 6.276) y SHOW (media = 6.948): $t = -23.3657$, $p < 0.001$.
- Puntuación z (scipy.stats.zscore) para detección de atípicos en tmdb_popularity: 61 títulos con $|z| > 3$ de 5 849 (1.04%).

Interpretación: la prueba de Shapiro-Wilk confirma que imdb_score no sigue una distribución normal, coherente con la ligera asimetría observada en el histograma. La correlación de Pearson entre calificación y popularidad es prácticamente nula y no significativa ($p > 0.05$): con esta muestra no hay evidencia de que un título mejor calificado sea, en promedio, más popular; ambas métricas capturan fenómenos distintos (calidad percibida vs. alcance de audiencia). En contraste, la prueba t de Welch sí muestra una diferencia altamente significativa: las series del catálogo están, en promedio, mejor calificadas que las películas (6.948 vs. 6.276). El z-score de SciPy confirma que solo alrededor de 1% de los títulos tiene una popularidad estadísticamente atípica frente al resto del catálogo.

5. Visualización con Matplotlib

Se elaboraron cuatro gráficas distintas con Matplotlib, pertinentes a las variables numéricas y categóricas seleccionadas:

5.1 Histograma de imdb_score

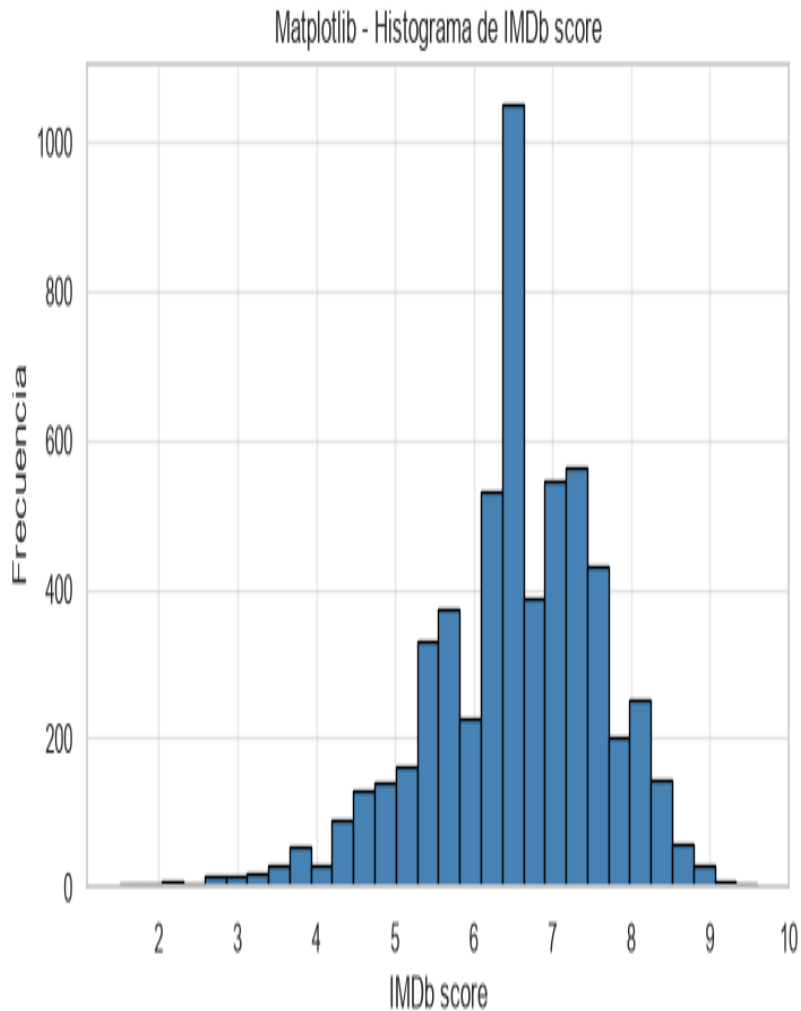


Figura 1. Distribución de IMDb score (Matplotlib).

Confirma visualmente lo detectado con NumPy: la mayoría de los títulos se concentra entre 5.5 y 7.5, con una cola hacia calificaciones bajas y otra, más corta, hacia calificaciones muy altas.

5.2 Dispersión: IMDb score vs. popularidad TMDb

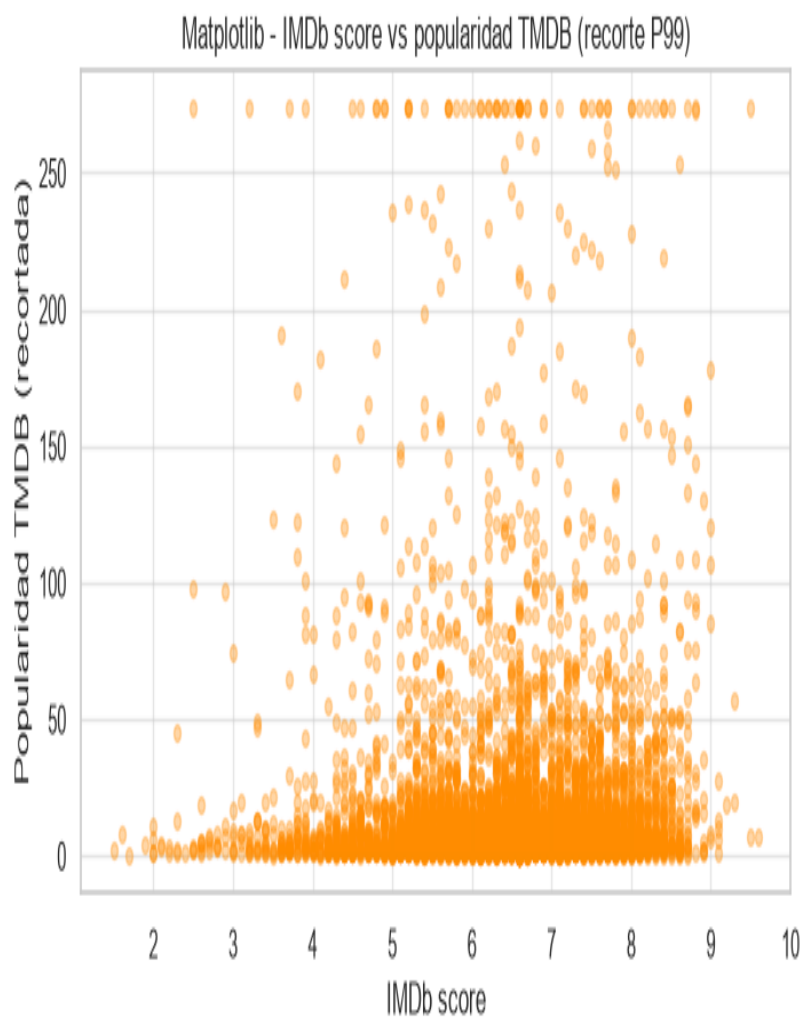


Figura 2. IMDb score vs. popularidad TMDb, recorte al P99 (Matplotlib).

La nube de puntos no muestra un patrón lineal claro entre calidad y popularidad, lo cual es coherente con la correlación de Pearson prácticamente nula calculada en SciPy.

5.3 Boxplot: imdb_score por tipo de contenido

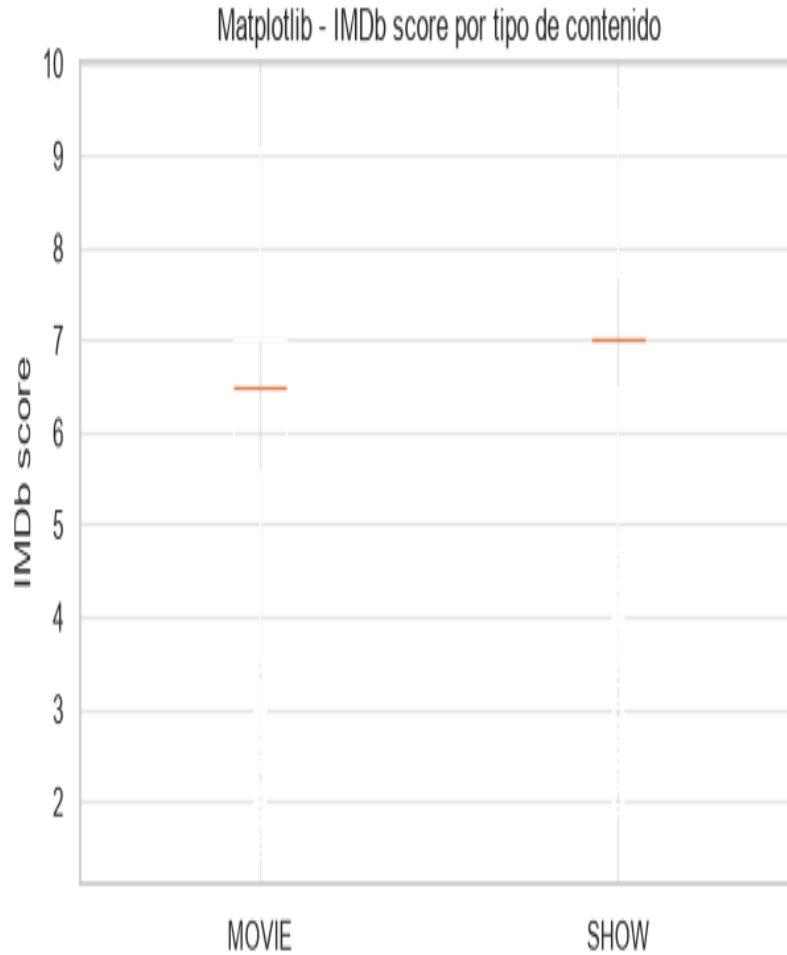


Figura 3. IMDb score por tipo de contenido (Matplotlib).

Las series (SHOW) muestran una mediana de calificación visiblemente más alta que las películas (MOVIE), con menor dispersión, lo que respalda la diferencia significativa detectada con la prueba t de Welch.

5.4 Barras: top 8 de géneros principales

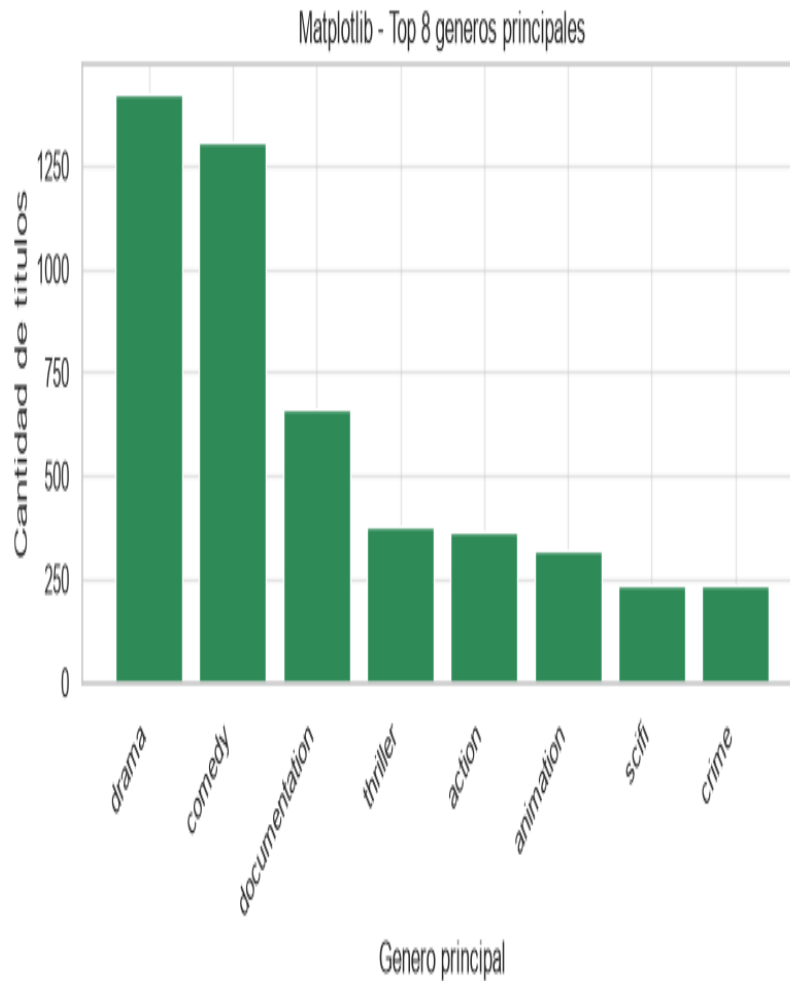


Figura 4. Top 8 de géneros principales (Matplotlib).

Se confirma visualmente el predominio de drama y comedia sobre el resto de géneros, seguidos por el documental, coherente con los conteos obtenidos con `np.unique`.

6. Visualización con Seaborn

Se elaboraron cuatro gráficas distintas con Seaborn, repitiendo y complementando el análisis visual anterior:

6.1 Distribución de imdb_score con curva de densidad (KDE)

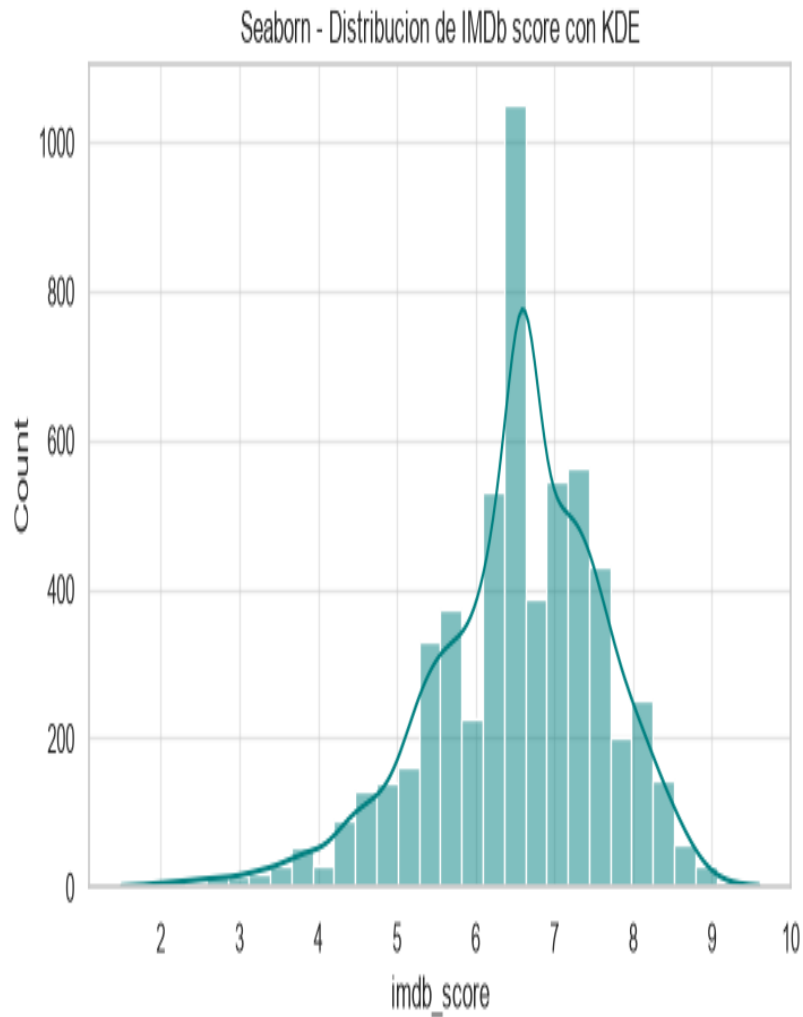


Figura 5. Distribución de IMDb score con KDE (Seaborn).

La curva de densidad superpuesta automáticamente por Seaborn hace más visible la ligera asimetría de la distribución, que Shapiro-Wilk ya había descartado como normal.

6.2 Boxplot: imdb_score por tipo de contenido

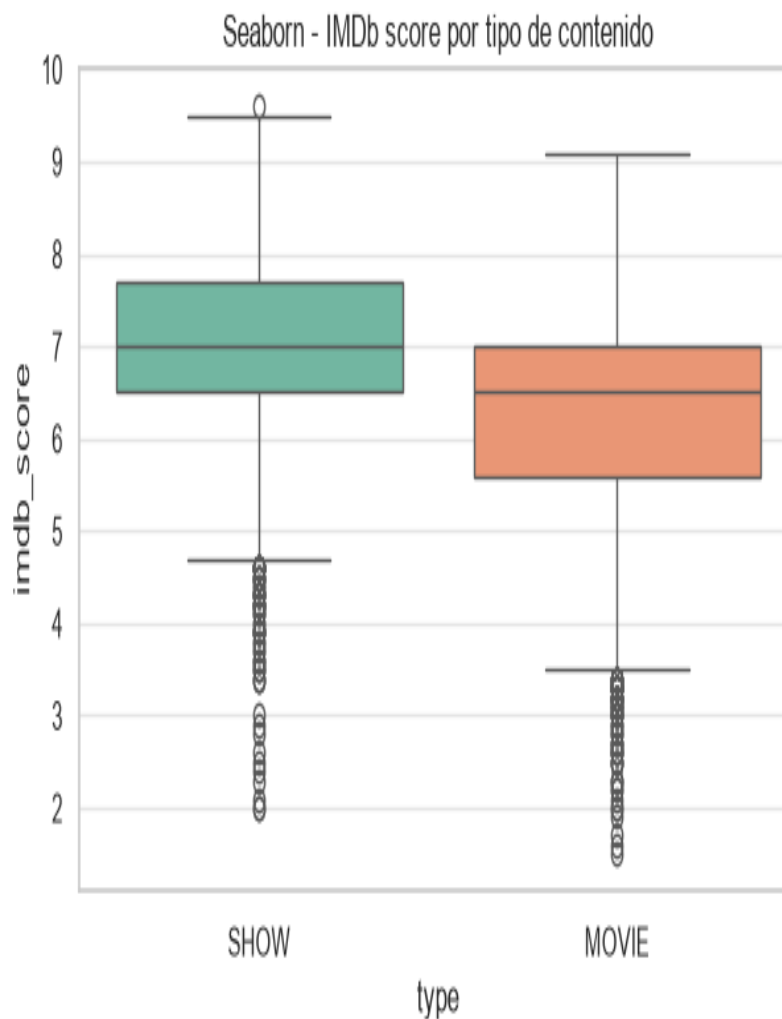


Figura 6. IMDb score por tipo de contenido (Seaborn).

Seaborn reproduce el mismo hallazgo que Matplotlib (mejor calificación en series) añadiendo color por categoría de forma automática.

6.3 Regplot: IMDb score vs. popularidad TMDB

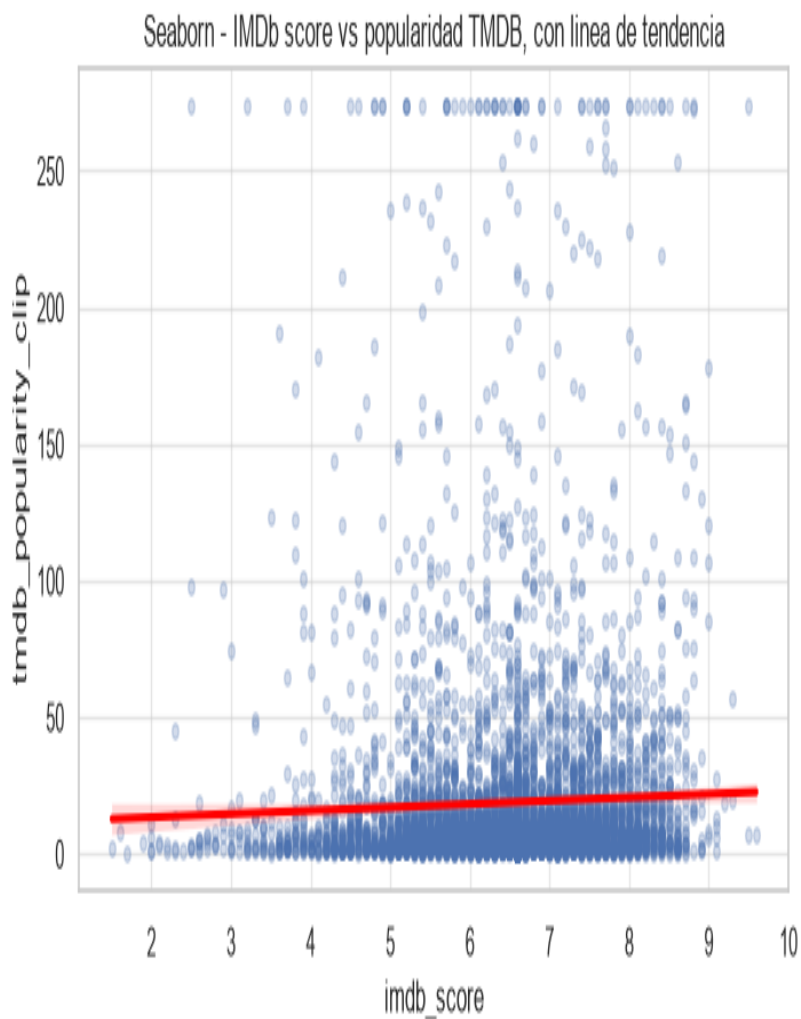


Figura 7. IMDb score vs. popularidad TMDB, con línea de tendencia (Seaborn).

regplot agrega automáticamente una línea de tendencia con intervalo de confianza; su pendiente casi plana confirma visualmente la correlación prácticamente nula ya cuantificada con Pearson.

6.4 Mapa de calor de correlación

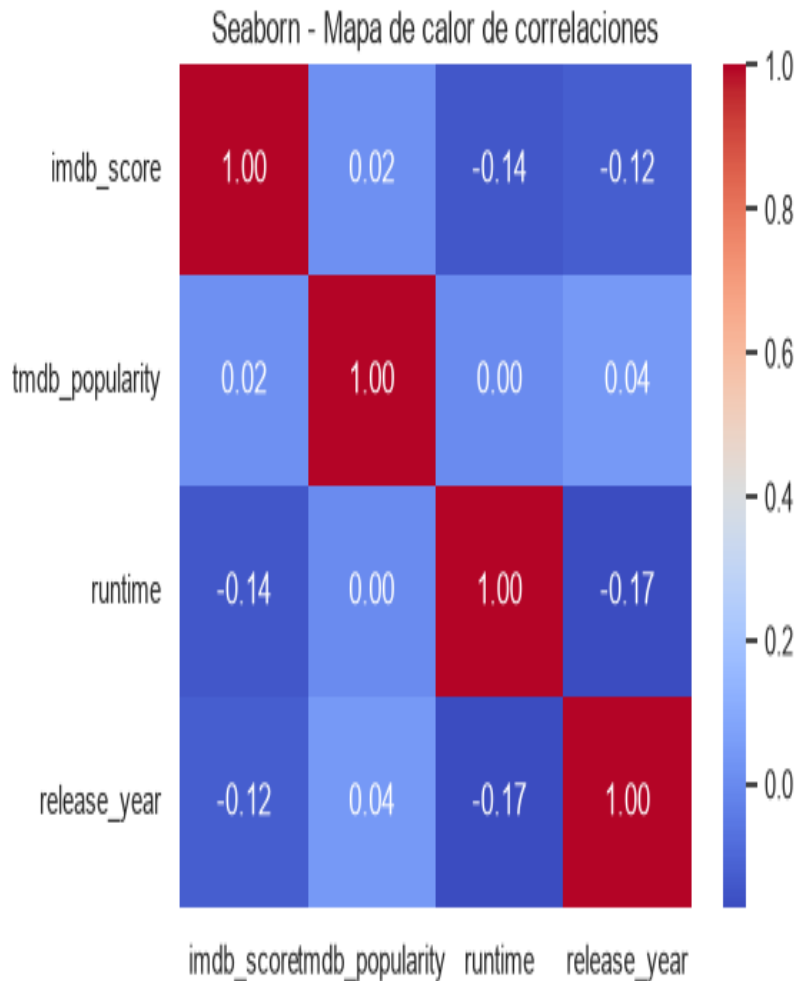


Figura 8. Correlación entre variables numéricas (Seaborn).

El heatmap resume en una sola imagen que ninguna de las correlaciones entre `imdb_score`, `tmdb_popularity`, `runtime` y `release_year` es fuerte, siendo todas cercanas a cero.

6.5 Comparación Matplotlib vs. Seaborn

Para el histograma y el scatterplot, Matplotlib exige definir manualmente cada elemento (bins, colores, título), mientras que Seaborn ofrece por defecto una estética más cuidada y funciones de alto nivel como `regplot` (línea de tendencia automática) o `histplot(kde=True)` (curva de densidad superpuesta) que en Matplotlib requerirían varias líneas adicionales. El heatmap de Seaborn, en particular, no tiene un equivalente directo tan simple en Matplotlib puro, donde habría que combinar `imshow` con anotaciones manuales de texto.

7. Respuestas a las preguntas del taller

7.1 ¿Qué fenómeno, proceso o contexto representa el dataset?

El dataset representa el catálogo de contenido audiovisual (películas y series) disponible en Netflix hasta 2022, incluyendo metadatos de producción (año, duración, país, género, clasificación por edad) y métricas externas de recepción de audiencia (calificación y popularidad en IMDb y TMDb), lo que permite estudiar patrones de composición del catálogo y de recepción del público.

7.2 ¿Cuáles son las 5 variables más importantes para describir el caso y por qué?

#	Variable	Tipo	Por qué es importante
1	type	Categórica	Distingue película de serie; el formato explica gran parte de la duración, la calificación y la estructura del catálogo.
2	imdb_score	Numérica	Mide la calidad percibida por la audiencia; es la variable central del análisis de NumPy, SciPy y de las visualizaciones.
3	tmdb_popularity	Numérica	Mide el alcance/interés generado por el título; complementa a imdb_score y permite comparar calidad vs. consumo.
4	release_year	Numérica	Permite entender la evolución temporal del catálogo y su crecimiento reciente.
5	primary_genre	Categórica (derivada)	Extraída de genres; segmenta el catálogo por tipo de contenido y explica parte de la variación en calificación y popularidad.

7.3 ¿Qué hallazgos se observan en la distribución de variables (categóricas y numéricas)?

En la variable categórica type existe un desbalance moderado: 64.0% de los títulos son películas frente a 36.0% de series. El género principal más frecuente es drama, seguido de comedia y documental. En las variables numéricas, imdb_score se concentra entre 5.5 y 7.5 y no pasa la prueba de normalidad de Shapiro-Wilk ($p < 0.001$); tmdb_popularity tiene una fuerte asimetría a la derecha (media = 22.4 muy por encima de la mediana = 6.8), con una pequeña cola de títulos extremadamente populares (1.04% con $|z| > 3$).

7.4 ¿Qué relación relevante encontraron entre las variables? (mínimo 2)

- imdb_score y tmdb_popularity prácticamente no están correlacionadas (Pearson $r = 0.0166$, $p = 0.2057$, no significativa): un título bien calificado no garantiza ser popular, y viceversa; calidad percibida y alcance de audiencia son fenómenos independientes en este catálogo.
- imdb_score difiere de forma estadísticamente significativa entre type (prueba t de Welch, $t = -23.37$, $p < 0.001$): las series tienen, en promedio, una calificación IMDb más alta que las películas (6.948 vs. 6.276).

7.5 ¿Qué problemas de calidad de datos detectaron y cómo los abordan?

Se detectaron nulos en age_certification (2 619, imputados como "No clasificado" por su alta proporción), nulos estructurales en seasons (3 744, que coinciden exactamente con el número de

películas, imputados con 0), nulos en las métricas de IMDb/TMDB (entre 91 y 498, imputados con la mediana para no perder observaciones) y 14 registros con runtime == 0, tratados como faltantes e imputados con la mediana según el tipo de contenido por ser una inconsistencia de captura y no un valor real. También se identificaron valores atípicos de alta popularidad en tmdb_popularity (59 títulos por encima del percentil 99), que se conservaron para el análisis estadístico pero se recortaron solo para efectos de visualización.

7.6 Con base en lo observado: ¿qué pregunta de IA o problema predictivo podría plantearse a futuro con este dataset?

Podría plantearse un problema de regresión para predecir imdb_score o tmdb_popularity a partir de variables como type, primary_genre, release_year y runtime, o un problema de clasificación para predecir si un título será "popular" (por ejemplo, superando cierto umbral de tmdb_popularity) usando esas mismas variables, útil para decisiones de adquisición o promoción de contenido. También sería posible un análisis de clustering para segmentar el catálogo en perfiles de contenido (por género, duración y calificación).

8. Pensamiento crítico

8.1 ¿Cuándo conviene usar Matplotlib y cuándo Seaborn?

Matplotlib conviene cuando se necesita control total sobre cada elemento de la gráfica —por ejemplo, el gráfico de barras de la sección 5.4, donde se definieron colores y rotación de etiquetas a la medida— o para gráficas simples y rápidas. Seaborn conviene cuando se trabaja directamente con un DataFrame y se quieren gráficas estadísticas ya elaboradas, como el heatmap de correlación (sección 6.4) o el regplot con línea de tendencia (sección 6.3), que en Matplotlib tomarían mucho más código.

8.2 ¿Qué ventajas aporta NumPy frente a trabajar solo con Pandas?

NumPy permite operaciones vectorizadas de bajo nivel —como la estandarización z-score de la sección 3— sin escribir ciclos for, con mejor rendimiento sobre arreglos grandes, y ofrece funciones numéricas (percentiles, unique con conteos) que Pandas reutiliza internamente. Además, muchas librerías de IA (scikit-learn, TensorFlow, PyTorch) reciben arreglos de NumPy como entrada estándar.

8.3 ¿Qué aportó SciPy que no era tan directo con NumPy/Pandas?

SciPy aportó pruebas estadísticas formales con su propio p-valor (Shapiro-Wilk para normalidad, correlación de Pearson con significancia, prueba t de Welch) que NumPy no calcula directamente, y funciones específicas como zscore que, de no usar SciPy, habría sido necesario programar manualmente con NumPy/Pandas.

8.4 Si el objetivo fuera construir un modelo de IA (predicción), ¿qué herramienta sumarían y por qué?

Se sumaría scikit-learn, ya que provee de forma directa algoritmos de clasificación y regresión, herramientas de partición de datos (`train_test_split`) y de evaluación de modelos, construidas justamente sobre los arreglos de NumPy y los DataFrames de Pandas ya preparados en este taller.

8.5 ¿Qué limitaciones detectaron en su dataset para aplicar análisis estadístico o visual?

El archivo `titles.csv` concentra una proporción importante de valores faltantes en `age_certification` (44.8%) y en las métricas de IMDb/TMDB (hasta 8.5%), lo que obliga a imputar en vez de trabajar solo con datos completos. Tampoco incluye información de consumo real dentro de la plataforma (solo métricas externas de IMDb/TMDB), y el catálogo podría estar sesgado hacia contenido reciente, lo que limita la generalización de cualquier hallazgo a años anteriores a 2015.

Referencias bibliográficas

Boschetti, A. y Massaron, L. (2018). Python Data Science Essentials: A Practitioner's Guide Covering Essential Data Science Principles, Tools, and Techniques (pp. 281-331). Packt Publishing Ltd.

Yim, A., Chung, C., y Yu, A. (2018). Matplotlib for python developers: Effective techniques for data visualization with python (2.^a ed., pp. 21-54). Packt Publishing, Limited.

Fuentes, A. (2018). Become a python data analyst: Perform exploratory data analysis and gain insight into scientific computing using python (pp. 69-118). Packt Publishing, Limited.

Mehta, H. (2015). Mastering Python Scientific Computing (pp. 163-198). Packt Publishing, Limited.

Soeiro, V. (2022). Netflix TV Shows and Movies. Kaggle. <https://www.kaggle.com/datasets/victorsoeiro/netflix-tv-shows-and-movies>

The SciPy community. (2024). SciPy documentation. <https://docs.scipy.org/doc/scipy/>